

3.1 LLM-Guided Evolutionary Search

The empirical code-optimization problem is difficult because the decision variable is not a vector in a Euclidean space, but an executable policy. A policy can be evaluated only by running it through the inventory simulator on the historical demand trajectories. This creates a black-box search problem over program space: small changes in code may lead to discontinuous changes in ordering behavior, and the empirical objective is non-differentiable with respect to the text of the program. We therefore solve **CODE-OPT** using an automatic heuristic design procedure inspired by Evolution of Heuristics (EoH) (Liu et al., 2024). EoH maintains a population of candidate heuristics, represents each heuristic by both natural-language reasoning and executable code, and uses LLM-based evolutionary prompts to generate new heuristics. We adopt this population-based view, but specialize it to stationary inventory policies and add a numerical tuning step for marked code parameters.

Policy representation. Each individual in the search population is a stationary inventory policy of the form introduced above:

$$a_t = \pi(I_t, Q_t).$$

Operationally, the policy is represented as a Python function that receives the current inventory state and returns an order quantity. We write a candidate individual as

$$z = (d, \pi, \iota, \hat{J}),$$

where d is a concise natural-language description of the policy, π is the executable code, ι is a short intuition explaining why the policy may be effective, and $\hat{J}$ is the empirical training cost. The description and intuition serve the same purpose as the “thought” component in EoH: they summarize the policy’s high-level logic in language, while the code specifies the exact decision rule that is evaluated in simulation. This representation allows the LLM to reason over both the conceptual policy idea and the concrete implementation when proposing a new candidate.

Initialization. The initialization step constructs the first surviving pool $\mathcal{P}^{(0)}$. In the original EoH framework, this can be done by an initialization prompt: the LLM is given the problem statement, the function interface, and the expected output format, but no parent heuristic, and is asked to propose initial heuristic policies from scratch. In our main implementation, however, we initialize the search with a standard base-stock policy. This gives the evolutionary process a feasible and interpretable starting point and provides a meaningful baseline for comparison. The base-stock initializer orders up to a target inventory position and can be written in the form

$$\pi_B(I_t, Q_t) = \max \left\{ 0, B - I_t - \sum_{k=1}^L q_{t,k} \right\},$$

where B is the base-stock level. If the base-stock level is marked as a tunable parameter and the external optimizer is enabled, it is optimized on the training trajectories before the first generation is formed.

Parent selection. At generation g , the algorithm maintains a surviving pool

$$\mathcal{P}^{(g)} = \{z_1^{(g)}, \dots, z_{K_g}^{(g)}\},$$

where each feasible candidate has already been evaluated on the training set. The candidates are ranked by empirical cost, with smaller $\hat{J}$ indicating better performance. Let m denote the number of parent policies provided to the LLM prompt. The parent set is denoted by

$$\mathcal{A}^{(g)} = \{z_{(1)}^{(g)}, \dots, z_{(m)}^{(g)}\} \subseteq \mathcal{P}^{(g)},$$

where $z_{(1)}^{(g)}, \dots, z_{(m)}^{(g)}$ are selected high-performing policies from the current surviving pool. In the main specification, $m = 1$, so each new candidate is generated by modifying one parent policy. In the multi-parent test, $m > 1$, so the same prompt template is supplied with several high-performing parents. This notation lets us describe both cases uniformly: the LLM always receives a parent set $\mathcal{A}^{(g)}$, and the size of that set determines whether the generation step is single-parent or multi-parent.

Prompt structure. The prompt is the interface between the evolutionary search and the LLM. Following the design principle of EoH, each prompt contains structured information rather than only a vague instruction to “write better code.” In our inventory setting, the prompt has five main components.

First, the *task description* states the lost-sales inventory problem, the planning and selling phases, the empirical objective, and the meaning of the policy inputs and output. This ensures that the LLM is generating an ordering rule for the correct simulator rather than a generic inventory heuristic. Second, the *parent-policy block* reports the selected parent set $\mathcal{A}^{(g)}$. For each parent, the prompt includes its description, executable code, intuition, and empirical cost statistics. Third, the *modification instruction* tells the LLM what kind of evolutionary operation to perform. In our main experiments this is the M2-style instruction: use the parent policy or parent policies as the starting point and produce a modified policy that is expected to reduce the average training cost. When multiple parents are provided, the same instruction naturally allows the LLM to combine useful components from different parents. Fourth, the *validity and stationarity restrictions* require the generated policy to depend only on the current state variables and to return a nonnegative order quantity. The prompt rules out hidden state, time counters, episode indices, global memory, and cached information across calls. Finally, the *output specification* requires the LLM response to contain a natural-language description, a complete Python implementation in the required function format, and a short intuition. If external parameter optimization is enabled, the prompt also explains how tunable constants should be marked using `OPT_PARAM` annotations.

The initialization prompt and the M2-style evolutionary prompt therefore serve different roles. The initialization prompt, when used, creates policies without parent information and is mainly a diversity-generating mechanism. The M2-style prompt is a performance-improvement mechanism: it conditions on the best policies found so far and asks the LLM to modify or synthesize a new stationary ordering rule. Our main model uses the base-stock initializer and the M2-style prompt with $m = 1$; the multi-parent experiment uses the same overall procedure with $m > 1$.

Optional parameter optimization. Inventory policies often contain numerical thresholds, safety-stock factors, pipeline weights, or smoothing constants. These constants are difficult for an LLM to tune precisely. To separate structural search from numerical tuning, the prompt asks the LLM to mark tunable constants using comments of the form

```
# OPT_PARAM: {'initial': v, 'min': l, 'max': u, 'type': 'float'}.
```

For a generated policy structure s , let θ_s denote the vector of marked constants and let Θ_s be the corresponding box constraints. If the generated code has no marked constants, it is evaluated

directly. If marked constants are present and the external optimizer is enabled, we solve the inner empirical tuning problem

$$\theta_s^* \in \arg \min_{\theta \in \Theta_s} \hat{J}(\pi_{s,\theta}),$$

where each objective call evaluates the resulting policy by simulation on the training trajectories. The unoptimized version is also evaluated, and the better version is kept. This step reduces the chance that a useful policy structure is discarded simply because its initial constants were poorly chosen by the LLM.

Evaluation and population update. After generation and optional tuning, the candidate policy is executed on the training trajectories to compute its empirical cost

$$\hat{J}(\pi) = \frac{1}{N} \sum_{n=1}^N C_n(\pi).$$

Policies that fail to parse, return invalid order quantities, time out, or otherwise cannot be evaluated are treated as infeasible and are excluded from the surviving pool. Feasible offspring are merged with the incumbent pool, duplicate candidates are removed using their empirical objective values, and the K unique policies with the lowest training costs are retained as $\mathcal{P}^{(g+1)}$. The procedure repeats for G generations and returns the feasible policy with the lowest empirical training cost.

Algorithmic summary. The full procedure is as follows.

1. **Initialize the surviving pool.** Load the base-stock policy as the initial candidate. Optionally, use the initialization prompt to generate additional initial policies. Evaluate all feasible initial policies on the training trajectories and retain the best candidates.
2. **Select parents.** At generation g , rank $\mathcal{P}^{(g)}$ by empirical cost and select a parent set $\mathcal{A}^{(g)}$ of size m . The main model uses $m = 1$; the multi-parent variant uses $m > 1$.
3. **Prompt the LLM.** Construct the M2-style prompt using the problem statement, parent-policy block, modification instruction, stationarity restrictions, parameter-marking rules, and required output format.
4. **Generate a candidate policy.** Parse the LLM response into a description, executable policy code, and intuition. Reject responses that do not contain valid executable policy code.
5. **Tune marked constants.** If the code contains `OPT_PARAM` annotations and external optimization is enabled, optimize the marked constants over their declared bounds while holding the policy structure fixed.
6. **Evaluate and update.** Simulate the resulting policy on the training set, compute $\hat{J}(\pi)$, merge feasible offspring with the incumbent pool, and retain the best K unique policies.
7. **Terminate.** After G generations, output the best feasible policy found.

Relation to EoH. Our procedure follows the core logic of EoH: heuristics are represented by both language and code; prompts act as evolutionary operators; candidate policies are evaluated on a fixed training set; and an elitist surviving pool preserves high-performing heuristics across generations. The key differences are summarized in Table 1. These differences reflect the inventory-control setting. Base-stock policies are strong, interpretable baselines, so we use one as the main initializer. In addition, because inventory heuristics frequently rely on continuous thresholds and scaling constants, we add an external optimizer to tune marked parameters after the LLM proposes a policy structure.

Table 1: Comparison between the original EoH framework and the inventory-control implementation in this paper.

Component	EoH	This paper
Individual representation	Natural-language thought, executable code, and fitness value.	Natural-language description, executable stationary inventory policy, intuition, and empirical cost.
Initialization	Initialization prompt generates initial heuristics from the task description.	Main implementation starts from a base-stock policy; initialization prompting can be used as an additional variant.
Evolution prompt	Multiple prompt strategies for exploration and modification.	M2-style modification prompt using a parent set $\mathcal{A}^{(g)}$ of size m .
Parent information	Parent heuristic descriptions and code are included in evolution prompts.	Parent policy descriptions, code, intuitions, and empirical cost statistics are included in the prompt.
Fitness evaluation	Run each heuristic on benchmark problem instances.	Simulate each policy on historical training demand trajectories.
Numerical tuning	Numerical constants are generated mainly by the LLM.	Marked constants are optionally tuned by a bounded continuous optimizer.

References

Liu, F., Tong, X., Yuan, M., Lin, X., Luo, F., Wang, Z., Lu, Z., and Zhang, Q. (2024). Evolution of heuristics: Towards efficient automatic algorithm design using large language model. In *Proceedings of the 41st International Conference on Machine Learning*.